

Day 2 - Complete Patient CRUD with Pagination, Custom Exceptions & Thymeleaf Forms

Date

2026-08-20

Overview

Day 2 focused on completing the Patient Management module by adding proper exception handling, pagination support, and full Thymeleaf web forms. The system now supports creating, editing, viewing, and deleting patients through both REST APIs and a web UI.

Technology Stack Used

Technology	Version	Purpose
Java	26	Programming Language
Spring Boot	4.1.0	Backend Framework
Spring Data MongoDB	4.1.0	Database Integration
MongoDB	Latest (local)	NoSQL Database
Thymeleaf	4.1.0	Server-side HTML templating
Bootstrap	5.3.2 (CDN)	UI Styling
Spring Validation	4.1.0	Request DTO validation
SpringDoc OpenAPI	2.8.0	API Documentation
Lombok	1.18.46	Boilerplate code reduction
Maven	Bundled with IntelliJ	Build & Dependency Management
Git	Latest	Version Control

What Was Built Today

1. Custom Exceptions (exception/)

`PatientNotFoundException.java`

- Extends `RuntimeException`
- Thrown when a patient ID does not exist in the database
- Replaces generic `RuntimeException` from Day 1

`DuplicateEmailException.java`

- Extends `RuntimeException`

- Thrown when attempting to create a patient with an email that already exists
- Ensures data integrity for patient email addresses

2. Updated Repository Layer (repository/)

PatientRepository.java - Added pagination methods:

- `findByNameContainingIgnoreCase(String name, Pageable pageable)` - Paginated name search
- `findAll(Pageable pageable)` - Paginated list of all patients
- Existing methods retained: `findByEmail`, `findByPhone`, `findByNameContainingIgnoreCase`

3. Updated Service Layer (service/)

PatientService.java - Interface expanded with:

- `getAllPatientsPaginated(int page, int size)` - Returns `Page<PatientResponseDto>`
- `searchPatientsByNamePaginated(String name, int page, int size)` - Paginated search

PatientServiceImpl.java - Key changes:

- `createPatient()` now checks for duplicate email via `findByEmail()`
- Throws `DuplicateEmailException` if email already exists
- `getPatientById()` now throws `PatientNotFoundException` instead of `RuntimeException`
- `deletePatient()` now throws `PatientNotFoundException` if ID not found
- `updatePatient()` now throws `PatientNotFoundException` if ID not found
- Added `getAllPatientsPaginated()` using `PageRequest.of(page, size)`
- Added `searchPatientsByNamePaginated()` using `PageRequest.of(page, size)`
- Manual `LocalDateTime.now()` for `createdAt` and `updatedAt` timestamps

4. Updated REST Controller (controller/)

PatientRestController.java - Added pagination support:

- `GET /api/patients` now accepts optional `page` and `size` query params
- `GET /api/patients/search?name=` now accepts optional `page` and `size` query params
- Returns `Page<PatientResponseDto>` when pagination params are provided
- Returns `List<PatientResponseDto>` when no pagination params (backward compatible)
- Uses wildcard return type `ResponseEntity<?>` for flexible response

5. Updated Thymeleaf Controller (controller/)

PatientViewController.java - Added full CRUD web operations:

- `GET /patients/new` - Shows create form
- `POST /patients/create` - Handles form submission, creates patient

- GET /patients/{id} - Shows patient detail page
- GET /patients/{id}/edit - Shows pre-filled edit form
- POST /patients/{id}/update - Handles form submission, updates patient
- GET /patients/{id}/delete - Deletes patient with confirmation
- Uses @ModelAttribute for form data binding
- Pre-populates edit form from existing patient data

6. New Thymeleaf Templates (templates/patients/ form.html - Patient

add/edit form:

- Bootstrap card layout
- Form fields: Name, Email, Phone, Gender (dropdown), Date of Birth, Blood Group (dropdown), Address
- Dynamic title: “Add New Patient” vs “Edit Patient”
- Dynamic submit button: “Create Patient” vs “Update Patient”
- Cancel button returns to patient list
- Uses th:object and th:field for Thymeleaf form binding detail.html - Patient detail view:
- Bootstrap card with info header showing Patient ID badge
- Table displaying all patient fields
- Formatted timestamps using #temporals.format
- Action buttons: Back to List, Edit, Delete
- Delete button with JavaScript confirmation dialog list.html (Updated) - Patient listing:
- Added “Actions” column with View, Edit, Delete buttons
- Added “+ Add Patient” button in header
- Updated REST API endpoint list in alert box to include pagination index.html (Updated) - Homepage:
- Added “Add Patient” button alongside “View Patients”
- Added dashboard card for Patients module
- Updated subtitle to reflect Day 2 features

Testing Performed

REST API Pagination Tests

1. Paginated List

```
GET http://localhost:8080/api/patients?page=0&size=2
```

- Result: Page object with content, totalElements, totalPages, number, size

2. Paginated Search

```
GET http://localhost:8080/api/patients/search?name=Ali&page=0&size=5
```

- Result: Paginated search results

Exception Tests

1. Duplicate Email

- POST patient with existing email
- Result: `DuplicateEmailException` thrown

2. Patient Not Found

- GET `/api/patients/PAT-NONEXISTENT`
- Result: `PatientNotFoundException` thrown

Thymeleaf UI Flow Tests

1. Create Patient via UI

- Navigate to `/patients/new`
- Fill form with all fields
- Submit -> Redirects to `/patients` with new patient in list

2. View Patient Details

- Click “View” on any patient row
- Shows detail page with all fields including formatted timestamps

3. Edit Patient via UI

- Click “Edit” on any patient row
- Form pre-filled with existing data
- Modify fields -> Submit -> Updated in list

4. Delete Patient via UI

- Click “Delete” on any patient row
- JavaScript confirmation dialog appears
- Confirm -> Patient removed from list

Challenges Faced & Solutions

Challenge	Solution
Pagination returning raw MongoDB documents	Used <code>Page.map()</code> to convert <code>Page<Patient></code> to <code>Page<PatientResponseDto></code>
Thymeleaf form pre-population for edit	Fetches existing patient, creates new <code>PatientRequestDto</code> , copies values, adds both to model
Date format display in Thymeleaf	Used <code>\${#temporals.format(patient.createdAt, 'yyyy-MM-dd HH:mm')}</code>
Form submission redirect after create/update	Returned <code>"redirect:/patients"</code> string from controller methods

Delete confirmation without JavaScript complexity

Used simple `onclick="return confirm('...')"` on anchor tag

Git Activity

- **Repository:** <https://github.com/abdul15x/clinic-management-system>
 - **Branch:** `main`
 - **Commit:** `feat: complete patient CRUD with pagination, custom exceptions, and Thymeleaf forms`
 - **Files pushed:** Updated service, controllers, repositories, new exception classes, new HTML templates
-

Files Created/Updated Today

New Files

```
src/main/java/com/clinic/exception/PatientNotFoundException.java
src/main/java/com/clinic/exception/DuplicateEmailException.java
src/main/resources/templates/patients/form.html
src/main/resources/templates/patients/detail.html
```

Updated Files

```
src/main/java/com/clinic/repository/PatientRepository.java
src/main/java/com/clinic/service/PatientService.java
src/main/java/com/clinic/service/PatientServiceImpl.java
src/main/java/com/clinic/controller/PatientRestController.java
src/main/java/com/clinic/controller/PatientViewController.java
src/main/resources/templates/patients/list.html
src/main/resources/templates/index.html
```

What's Next (Day 3 Preview)

- Doctor Management module
 - `Doctor` entity with specialization enum
 - Doctor CRUD REST APIs
 - Doctor search by specialization
 - Thymeleaf doctor list and form pages
-

How to Run This Version

1. Ensure MongoDB is running on `localhost:27017`
2. Open project in IntelliJ IDEA
3. Set Project SDK to Java 26
4. Enable Annotation Processing

5. Run `ClinicManagementSystemApplication.java`

6. Access:

- **Web UI:** `http://localhost:8080/`
- **Patient List:** `http://localhost:8080/patients`
- **Add Patient:** `http://localhost:8080/patients/new`
- **REST API:** `http://localhost:8080/api/patients`